

Exercise 5.1 Mini-Calc

The syntax of language *Mini-Calc* is as follows:

$$e ::= \begin{array}{l} 1 \\ | \text{ "Uhhh" } \\ | e + e \\ | \text{ str}(e) \\ | \text{ print}(e) \\ | e, e \end{array}$$

Ensure that numbers may be added to numbers and strings to strings. However, strings may not be added on to numbers and vice versa. `str` can convert a number to a string. `print` expects strings and returns a number representing the status code. e_1, e_2 has the type of expression e_2 , but e_1 still needs to be either a number or a string.

Define a mathematically precise typing relation for this language using rules of inference.

Exercise 5.2 C++: Undecidable Type Checking

Write a small C++ program that demonstrates why type checking in C++ is undecidable. Your program should encode a computation inside C++ template instantiation such that the compiler must evaluate that computation *during type checking*. Construct the program so that template instantiation does not terminate (or forces the compiler to hit its instantiation-depth limit). Use only standard C++ templates; no extensions.

Exercise 5.3 Simply Typed λ -Calculus

Is there any context Γ and type T such that $\Gamma \vdash x x : T$? If so, give Γ and T and show a typing derivation for $\Gamma \vdash x x : T$; if not, prove it.